Bayesian Logistic Regression - Flight Delay Prediction

2026-04-12

Overview

This file implements a Bayesian logistic regression model to predict whether a US domestic flight will be delayed by 15 or more minutes from initial departure. The model is fit using a manual Metropolis sampler in R, results are compared against the MLP model (ROC-AUC 0.7146) built in Python.

The dataset is a stratified 50,000-row subsample of the BTS On-Time Performance data (January–December 2025), preprocessed in Python and exported as `bay_sample_50k.csv`.

```r
library(ggplot2)
library(pROC)
library(data.table)
library(nestedcv)
```

```r
df <- read.csv("data/bay_sample_50k.csv")
str(df)
```

```
## 'data.frame':    50000 obs. of  9 variables:
##  $ MONTH            : int  7 9 5 6 4 10 7 4 7 3 ...
##  $ DAY_OF_MONTH     : int  4 13 21 1 30 8 18 11 18 2 ...
##  $ DAY_OF_WEEK      : int  5 6 3 7 3 3 5 5 5 7 ...
##  $ OP_UNIQUE_CARRIER: chr  "WN" "DL" "HA" "B6" ...
##  $ ORIGIN           : chr  "Other" "ATL" "SFO" "Other" ...
##  $ DEST             : chr  "ATL" "Other" "Other" "MCO" ...
##  $ DEP_DEL15        : int  0 0 0 0 1 0 0 1 0 0 ...
##  $ DISTANCE         : num  696 577 2338 944 354 ...
##  $ DEP_HOUR         : int  8 15 8 7 18 21 18 12 17 17 ...
```

```r
head(df)
```

```
##   MONTH DAY_OF_MONTH DAY_OF_WEEK OP_UNIQUE_CARRIER ORIGIN  DEST DEP_DEL15
## 1     7            4           5                WN  Other   ATL         0
## 2     9           13           6                DL    ATL Other         0
## 3     5           21           3                HA    SFO Other         0
## 4     6            1           7                B6  Other   MCO         0
## 5     4           30           3                MQ  Other   DFW         1
## 6    10            8           3                WN  Other Other         0
##   DISTANCE DEP_HOUR
## 1      696        8
## 2      577       15
## 3     2338        8
## 4      944        7
## 5      354       18
## 6      452       21
```

Preprocessing

ORIGIN, DEST, OP_UNIQUE_CARRIER, MONTH, and DAY_OF_WEEK are one-hot encoded. These features are encoded rather than treated as numeric given the non-linear relationships identified in EDA. DAY_OF_MONTH, DEP_HOUR, and DISTANCE are standardized using `scale()` fit on the training set only to prevent leakage. A stratified split is used instead of simple random sampling to preserve the 78/22 class balance. An intercept column is added explicitly.

```r
cat_cols <- c("ORIGIN", "DEST", "OP_UNIQUE_CARRIER", "MONTH", "DAY_OF_WEEK")
for (col in cat_cols) df[[col]] <- as.factor(df[[col]])

# Stratified 80/20 train/test split on DEP_DEL15
set.seed(42)
delayed_idx <- which(df$DEP_DEL15 == 1)
ontime_idx  <- which(df$DEP_DEL15 == 0)

train_idx <- c(
  sample(delayed_idx, floor(0.8 * length(delayed_idx))),
  sample(ontime_idx,  floor(0.8 * length(ontime_idx)))
)
test_idx <- setdiff(1:nrow(df), train_idx)

train_df <- df[train_idx, ]
test_df  <- df[test_idx,  ]

cat("Train size:", nrow(train_df), "| Delay rate:", round(mean(train_df$DEP_DEL15), 4), "\n")
```

```
## Train size: 39999 | Delay rate: 0.2169
```

```r
cat("Test size: ", nrow(test_df),  "| Delay rate:", round(mean(test_df$DEP_DEL15),  4), "\n")
```

```
## Test size:  10001 | Delay rate: 0.217
```

```r
# Separate target
y_train <- train_df$DEP_DEL15
y_test  <- test_df$DEP_DEL15

train_df$DEP_DEL15 <- NULL
test_df$DEP_DEL15  <- NULL

# One-hot encode following class approach
X_train <- as.matrix(nestedcv::one_hot(as.data.table(train_df)))
X_test  <- as.matrix(nestedcv::one_hot(as.data.table(test_df)))

# Align test columns to train (handles unseen levels in test)
missing_cols <- setdiff(colnames(X_train), colnames(X_test))
for (col in missing_cols) X_test <- cbind(X_test,
                                          setNames(data.frame(rep(0, nrow(X_test))), col))
X_test <- X_test[, colnames(X_train)]

# Standardize numeric features - fit on train only, applied to test
num_cols <- c("DAY_OF_MONTH", "DEP_HOUR", "DISTANCE")
train_means <- colMeans(X_train[, num_cols])
train_sds   <- apply(X_train[, num_cols], 2, sd)

X_train[, num_cols] <- scale(X_train[, num_cols])
```

```r
X_test[, num_cols]  <- scale(X_test[, num_cols], center = train_means, scale = train_sds)

# Add intercept column - class convention
X_train <- cbind(bias = 1, X_train)
X_test  <- cbind(bias = 1, X_test)

cat("Feature count:", ncol(X_train), "\n")
```

```
## Feature count: 74
```

Model Specification

The model follows Bayesian logistic regression. The likelihood is Bernoulli with a sigmoid link function. The prior on all coefficients is N(0, 10) placing no strong belief on the direction or magnitude of any feature's effect. The log-posterior is the sum of log-likelihood and log-prior, computed in log space throughout to avoid numerical underflow with 50,000 observations.

```r
sigmoid <- function(z) 1 / (1 + exp(-z))

log_likelihood <- function(beta, X, y) {
  p <- sigmoid(X %*% beta)
  sum(y * log(p + 1e-10) + (1 - y) * log(1 - p + 1e-10))
}

log_prior <- function(beta, prior_sd = 10) {
  sum(dnorm(beta, mean = 0, sd = prior_sd, log = TRUE))
}

log_posterior <- function(beta, X, y) {
  log_likelihood(beta, X, y) + log_prior(beta)
}
```

Metropolis Sampler

he proposal standard deviation sigma was tuned via pilot runs to achieve an acceptance rate within the target range of 20–40%. The sampler uses an isotropic Gaussian proposal, log-space acceptance condition, and a single vector update of beta_current.

```r
metropolis_bayes <- function(X, y, n_iter = 5000, sigma = 0.01, burn = 0.18) {
  p <- ncol(X)
  beta_current <- rep(0, p)
  samples <- matrix(0, nrow = n_iter, ncol = p)
  accepted <- 0

  for (i in 1:n_iter) {
    beta_prop <- rnorm(p, mean = beta_current, sd = sigma)
    log_alpha <- log_posterior(beta_prop, X, y) - log_posterior(beta_current, X, y)

    if (log(runif(1)) < log_alpha) {
      beta_current <- beta_prop
      accepted <- accepted + 1
    }
    samples[i, ] <- beta_current
  }
```

```r
  cat("Acceptance rate:", round(accepted / n_iter, 3), "\n")
  burn_n <- floor(burn * n_iter)
  return(samples[(burn_n + 1):n_iter, ])
}

# pilot run to tune sigma: 500 iterations burn = 0 to inspect acceptance rate
#set.seed(42)
#tuning <- metropolis_bayes(X_train, y_train, n_iter = 500, sigma = 0.03, burn = 0)
```

Full Metropolis Run

```r
set.seed(42)
samples <- metropolis_bayes(X_train, y_train, n_iter = 100000, sigma = 0.01,
                            burn = 0.18)
```

```
## Acceptance rate: 0.27
```

```r
cat("Posterior samples shape:", nrow(samples), "x", ncol(samples), "\n")
```

```
## Posterior samples shape: 82000 x 74
```

A sigma of 0.01 was selected after a few test runs, yielding an acceptance rate of 0.27 on the full run within the target range of 20–40%

Convergence Checks

Trace plots and ACF plots are examined for three key coefficients — DEP_HOUR, DISTANCE, and one carrier

```r
col_names <- colnames(X_train)
idx_hour     <- which(col_names == "DEP_HOUR")
idx_distance <- which(col_names == "DISTANCE")
idx_carrier  <- which(grepl("OP_UNIQUE_CARRIER", col_names))[1]

cat("DEP_HOUR index:  ", idx_hour, "\n")
```

```
## DEP_HOUR index:   4
```

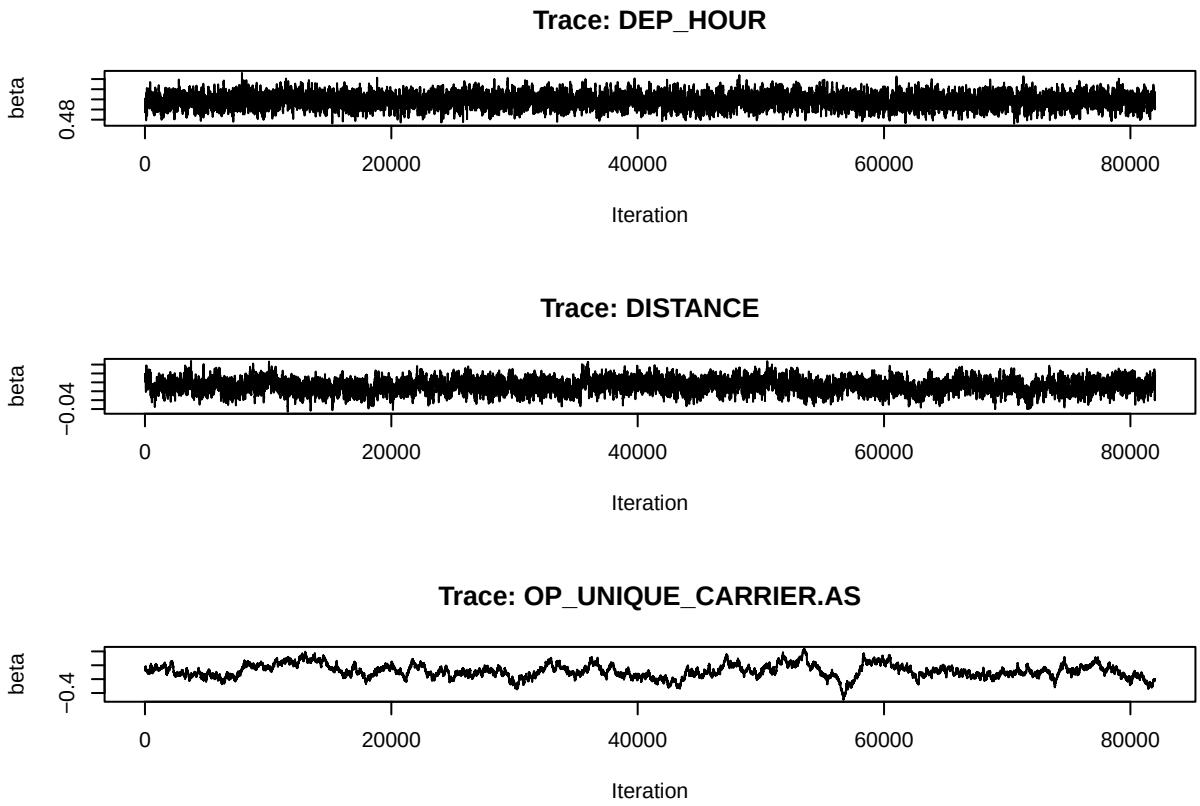
```r
cat("DISTANCE index:  ", idx_distance, "\n")
```

```
## DISTANCE index:   3
```

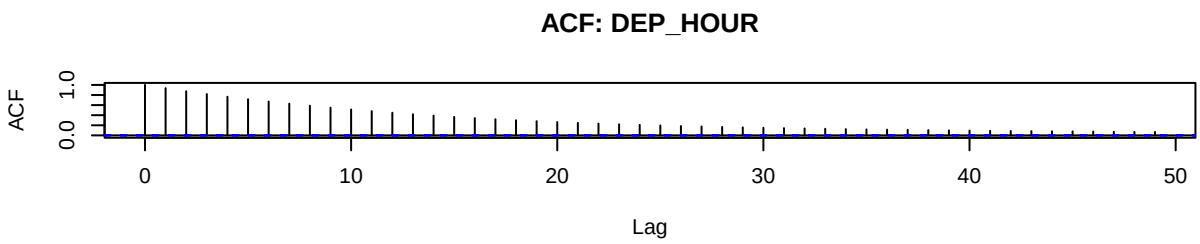
```r
cat("Carrier:         ", col_names[idx_carrier], "\n")
```

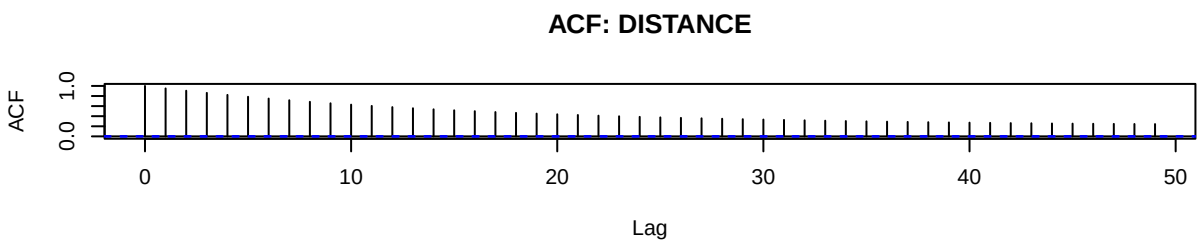
```
## Carrier:          OP_UNIQUE_CARRIER.AS
```

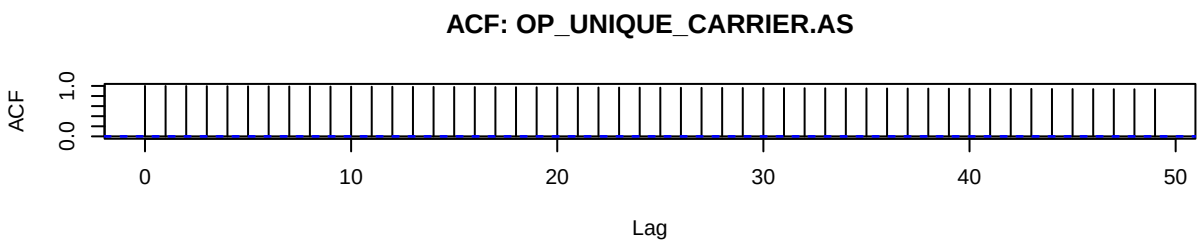
```r
# Trace plots
par(mfrow = c(3, 1))
plot(samples[, idx_hour],     type = "l", main = "Trace: DEP_HOUR",
     ylab = "beta", xlab = "Iteration")
plot(samples[, idx_distance], type = "l", main = "Trace: DISTANCE",
     ylab = "beta", xlab = "Iteration")
plot(samples[, idx_carrier],  type = "l", main = paste("Trace:", col_names[idx_carrier]),
     ylab = "beta", xlab = "Iteration")
```




```r
# ACF plots
par(mfrow = c(3, 1))
acf(samples[, idx_hour],     main = "ACF: DEP_HOUR")
acf(samples[, idx_distance], main = "ACF: DISTANCE")
acf(samples[, idx_carrier],  main = paste("ACF:", col_names[idx_carrier]))
```








```r
thinned_samples <- samples[seq(1, nrow(samples), by = 50), ]
colnames(thinned_samples) <- colnames(X_train)
cat("Thinned samples:", nrow(thinned_samples), "\n")
```

```
## Thinned samples: 1640
```

```r
col_names <- colnames(X_train)
idx_hour     <- which(col_names == "DEP_HOUR")
idx_distance <- which(col_names == "DISTANCE")
idx_carrier  <- which(grepl("OP_UNIQUE_CARRIER", col_names))[1]

cat("DEP_HOUR index:  ", idx_hour, "\n")
```

```
## DEP_HOUR index:   4
```

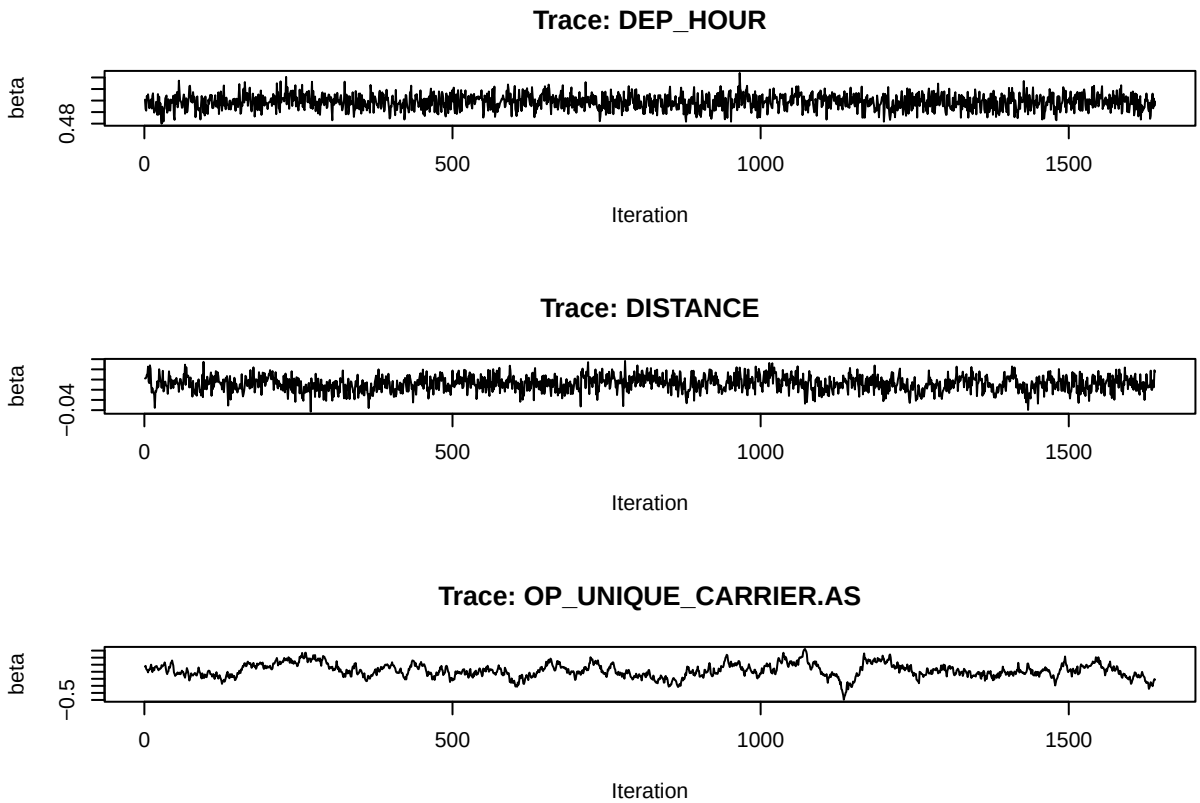
```r
cat("DISTANCE index:  ", idx_distance, "\n")
```

```
## DISTANCE index:   3
```

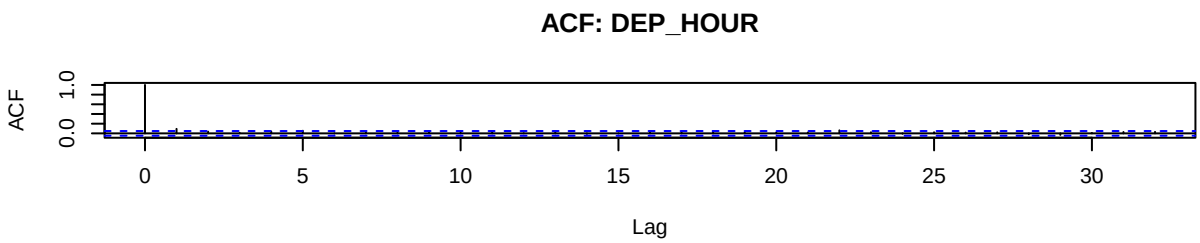
```r
cat("Carrier:         ", col_names[idx_carrier], "\n")
```

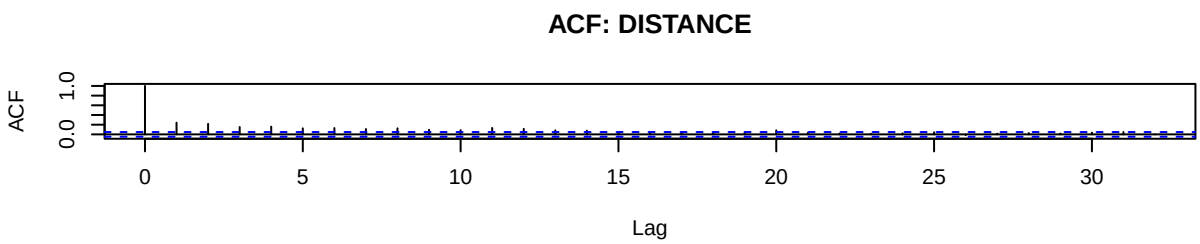
```
## Carrier:          OP_UNIQUE_CARRIER.AS
```

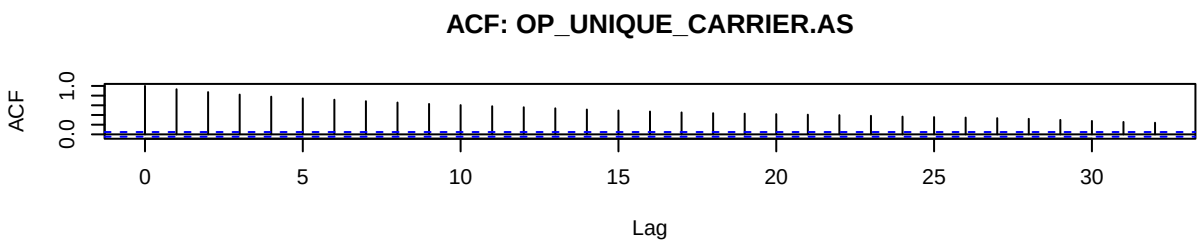
```r
# Trace plots
par(mfrow = c(3, 1))
plot(thinned_samples[, idx_hour],     type = "l", main = "Trace: DEP_HOUR",
     ylab = "beta", xlab = "Iteration")
plot(thinned_samples[, idx_distance], type = "l", main = "Trace: DISTANCE",
     ylab = "beta", xlab = "Iteration")
plot(thinned_samples[, idx_carrier],  type = "l", main = paste("Trace:", col_names[idx_carrier]),
     ylab = "beta", xlab = "Iteration")
```




```r
# ACF plots
par(mfrow = c(3, 1))
acf(thinned_samples[, idx_hour],     main = "ACF: DEP_HOUR")
acf(thinned_samples[, idx_distance], main = "ACF: DISTANCE")
acf(thinned_samples[, idx_carrier],  main = paste("ACF:", col_names[idx_carrier]))
```








```r
# Posterior mean betas as point estimates using thinned samples
beta_hat <- colMeans(thinned_samples)

# Predicted probabilities on test set
p_test <- sigmoid(X_test %*% beta_hat)

# ROC-AUC
roc_obj <- roc(y_test, as.vector(p_test), quiet = TRUE)
cat("Bayesian Logistic Regression ROC-AUC:", round(auc(roc_obj), 4), "\n")
```

```
## Bayesian Logistic Regression ROC-AUC: 0.6694
```

```r
cat("MLP ROC-AUC (baseline):               0.7146\n")
```

```
## MLP ROC-AUC (baseline):               0.7146
```

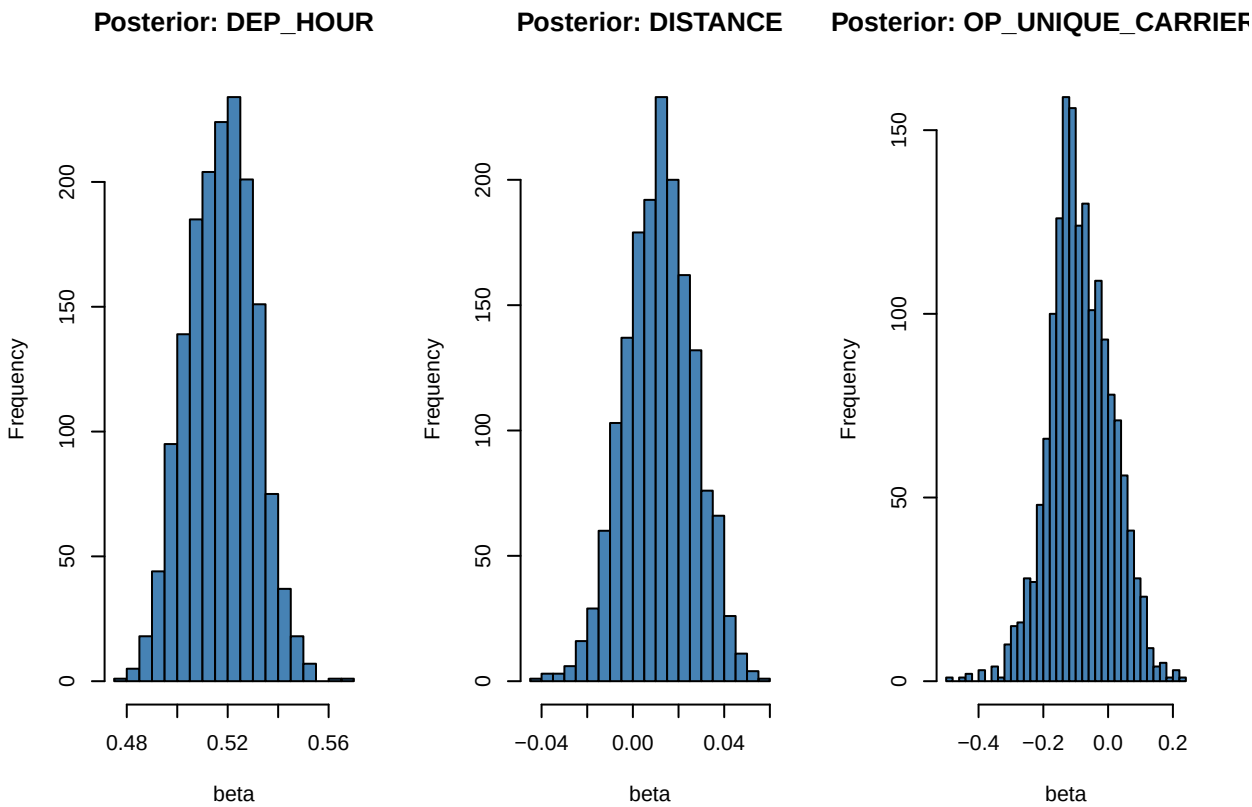
```r
# 95% credible intervals for key coefficients
ci <- apply(thinned_samples, 2, quantile, c(0.025, 0.975))
key_cols <- c("DEP_HOUR", "DISTANCE", col_names[idx_carrier])
cat("\n95% Credible Intervals:\n")
```

```
##
## 95% Credible Intervals:
```

```r
print(round(ci[, key_cols], 4))
```

```
##       DEP_HOUR DISTANCE OP_UNIQUE_CARRIER.AS
## 2.5%    0.4915  -0.0181              -0.2772
## 97.5%   0.5432   0.0401               0.1052
```

```r
# Posterior histograms
par(mfrow = c(1, 3))
hist(thinned_samples[, idx_hour],     breaks = 30, main = "Posterior: DEP_HOUR",
     xlab = "beta", col = "steelblue")
hist(thinned_samples[, idx_distance], breaks = 30, main = "Posterior: DISTANCE",
     xlab = "beta", col = "steelblue")
hist(thinned_samples[, idx_carrier],  breaks = 30,
     main = paste("Posterior:", col_names[idx_carrier]),
     xlab = "beta", col = "steelblue")
```




```r
par(mfrow = c(1, 1))
```